



Future Interns

Vulnerability Assessment Report for a Live Website

Target Website:

<http://testphp.vulnweb.com>

Assessment Type:

Read-Only Web Application Vulnerability Assessment

Prepared by:

[SN Amogh](#) (FIT/FEB26/CS6277)

Intern at [Future Interns](#)

Tools Used:

Nmap, OWASP ZAP (Passive Scan), Browser Developer Tools

Confidentiality Note:

This report contains the results of a security assessment conducted on the target web application.

All testing activities were conducted in a read-only manner without exploiting vulnerabilities or disrupting system functionality.

The information presented in this report should be used responsibly and only for improving the security posture of the evaluated system.

Assessment Overview:

This report presents the results of a vulnerability assessment performed on the website <http://testphp.vulnweb.com>.

The objective of this assessment was to identify potential security weaknesses using passive testing techniques and publicly available security tools. The evaluation focused on identifying common misconfigurations, exposed services, and web application vulnerabilities.

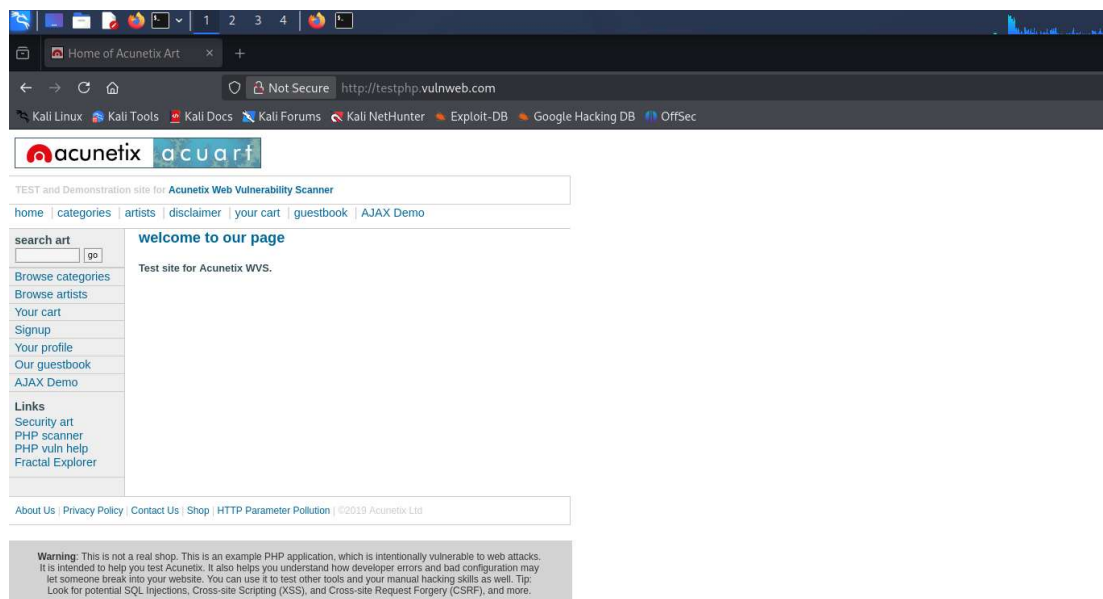
Several tools were used during the assessment including Kali Linux, Nmap, OWASP ZAP, browser developer tools, and DNS reconnaissance utilities.

Appropriate security recommendations have been provided to help improve the overall security posture of the application.

Target Verification:

Before beginning the assessment, the target website was accessed to confirm availability and observe basic application behavior.

The website <http://testphp.vulnweb.com> is a deliberately vulnerable application used for demonstrating web application security testing.



The browser also indicated that the website uses HTTP instead of HTTPS, which means communication between the client and server is not encrypted.

Scope & Methodology:

Assessment Scope

The scope of this security assessment was limited to a non-intrusive, read-only evaluation of the publicly accessible components of the target website. The objective of the assessment was to identify potential security weaknesses and configuration issues without performing active exploitation or impacting system functionality.

The assessment included the following activities:

- Review of publicly accessible web pages
- Inspection of HTTP response headers
- Passive web application vulnerability scanning
- Network service visibility checks

The following activities were intentionally excluded from this assessment:

- Authentication bypass attempts
- Unauthorized data access or modification
- Denial-of-Service (DoS) testing
- Active exploitation of identified vulnerabilities

Assessment Methodology

The security assessment was conducted using commonly accepted web security testing practices, focusing on passive analysis and configuration inspection.

Industry-standard security tools were used to identify potential weaknesses in the application infrastructure and response behaviour.

The tools and techniques used during the assessment included:

- Nmap – for network port scanning and service detection
- OWASP ZAP (Passive Scan) – for identifying common web application vulnerabilities
- Browser Developer Tools – for inspecting HTTP headers, cookies, and client-side behaviour

All findings presented in this report are based on observable system responses and automated tool-generated evidence collected during the time of testing.

Environment Setup:

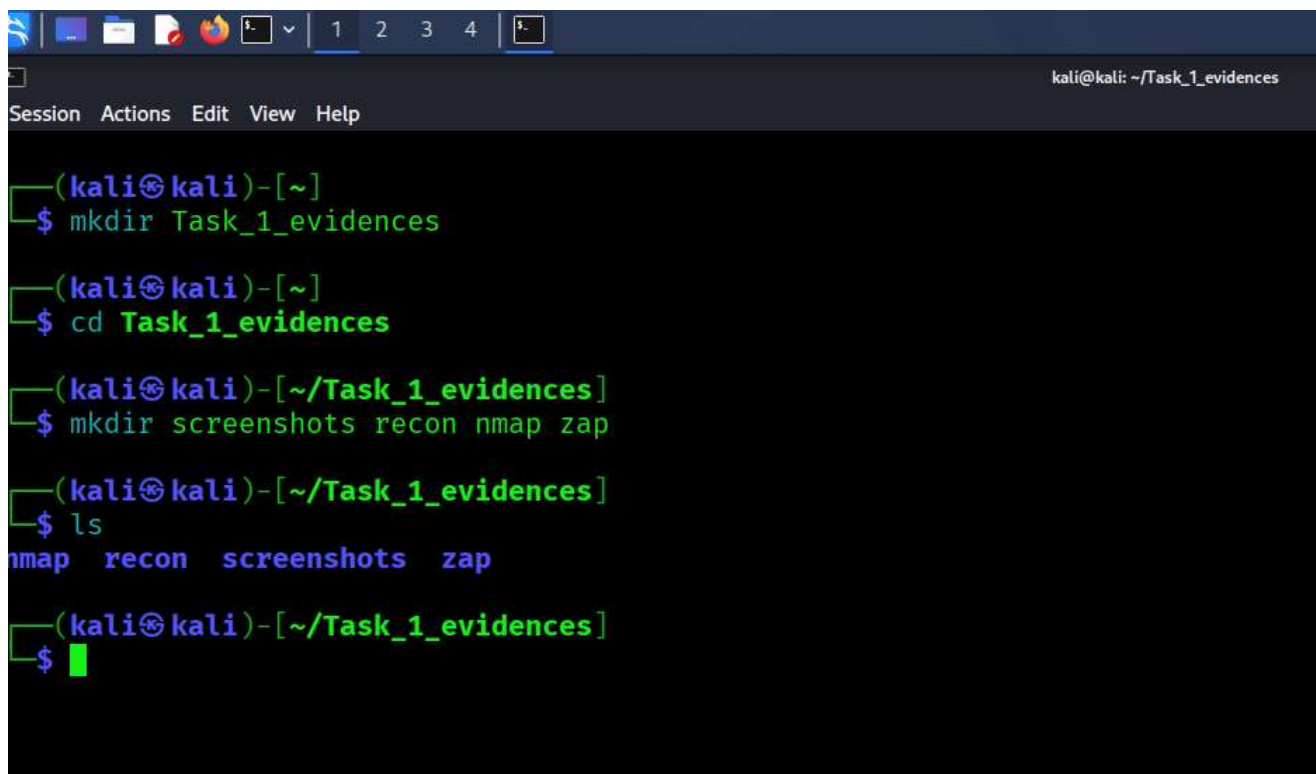
A dedicated working directory was created in Kali Linux to store evidence collected during the assessment process.

The evidence directory was structured to organize screenshots and outputs according to the testing phase.

Directory Structure:

Task_1_evidences

- Screenshots
- Recon
- Nmap
- Zap

A terminal window screenshot from Kali Linux. The window title is 'kali@kali: ~/Task_1_evidences'. The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ mkdir Task_1_evidences  
  
(kali@kali)-[~]  
$ cd Task_1_evidences  
  
(kali@kali)-[~/Task_1_evidences]  
$ mkdir screenshots recon nmap zap  
  
(kali@kali)-[~/Task_1_evidences]  
$ ls  
nmap  recon  screenshots  zap
```

Reconnaissance:

Reconnaissance was performed to gather publicly available information about the target domain and hosting infrastructure.

Information collected during this phase included DNS resolution, domain registration details, and server response headers.

This information helps security analysts understand how the application is hosted and identify potential entry points for further analysis.

```
kali@kali: /
Session Actions Edit View Help

(kali@kali)-[/]
$ whois vulnweb.com
Domain Name: VULNWEB.COM
Registry Domain ID: 1602006391_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.gandi.net
Registrar URL: http://www.gandi.net
Updated Date: 2025-11-17T09:34:20Z
Creation Date: 2010-06-14T07:50:29Z
Registry Expiry Date: 2027-06-14T07:50:29Z
Registrar: Gandi SAS
Registrar IANA ID: 81
Registrar Abuse Contact Email: abuse@support.gandi.net
Registrar Abuse Contact Phone: +33.170377661
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: NS-105-A.GANDI.NET
Name Server: NS-11-B.GANDI.NET
Name Server: NS-140-C.GANDI.NET
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-03-04T11:54:46Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.
```

```
kali@kali: /
Session Actions Edit View Help

The Registry database contains ONLY .COM, .NET, .EDU domains and
Registrars.
Domain Name: vulnweb.com
Registry Domain ID: 1602006391_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.gandi.net
Registrar URL: http://www.gandi.net
Updated Date: 2025-11-17T09:34:23Z
Creation Date: 2010-06-14T05:50:29Z
Registrar Registration Expiration Date: 2027-06-14T07:50:29Z
Registrar: GANDI SAS
Registrar IANA ID: 81
Registrar Abuse Contact Email: abuse@support.gandi.net
Registrar Abuse Contact Phone: +33.170377661
Reseller:
Domain Status: clientTransferProhibited http://www.icann.org/epp#clientTransferProhibited
Domain Status:
Domain Status:
Domain Status:
Domain Status:
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: Invicti Security Limited
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province:
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: MT
Registrant Phone: REDACTED FOR PRIVACY
Registrant Phone Ext:
Registrant Fax: REDACTED FOR PRIVACY
Registrant Fax Ext:
```

DNS Resolution:

DNS lookup was performed to identify the IP address associated with the target domain.

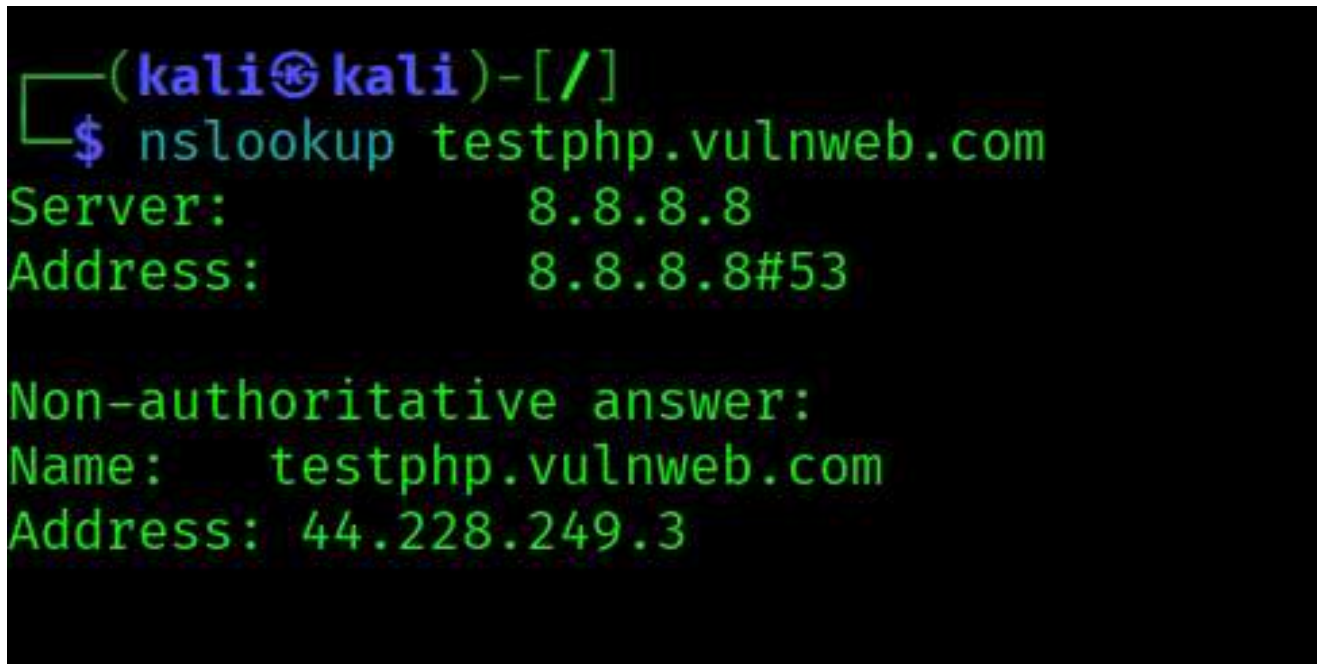
Command Used:

```
nslookup testphp.vulnweb.com
```

Result:

Domain resolved to IP address:

44.228.249.3

A terminal window with a black background and green text. The prompt is (kali@kali)-[//]. The command nslookup testphp.vulnweb.com has been entered. The output shows the server 8.8.8.8 and address 8.8.8.8#53. Below this, it says 'Non-authoritative answer:' followed by 'Name: testphp.vulnweb.com' and 'Address: 44.228.249.3'.

```
(kali@kali)-[//]  
$ nslookup testphp.vulnweb.com  
Server:                8.8.8.8  
Address:                8.8.8.8#53  
  
Non-authoritative answer:  
Name:   testphp.vulnweb.com  
Address: 44.228.249.3
```

This confirms that the website is hosted on an externally accessible server.

Domain Information Discovery:

WHOIS analysis was conducted to obtain domain registration information.

Command Used: *whois vulnweb.com*

The WHOIS records revealed the following information:

- Domain Name: vulnweb.com
- Registrar: Gandi SAS
- Creation Date: June 2010
- Expiration Date: June 2027

```
kali@kali: /  
--(kali@kali)-[/]  
$ whois vulnweb.com  
Domain Name: VULNWEB.COM  
Registry Domain ID: 1602006391_DOMAIN_COM-VRSN  
Registrar WHOIS Server: whois.gandi.net  
Registrar URL: http://www.gandi.net  
Updated Date: 2025-11-17T09:34:20Z  
Creation Date: 2010-06-14T07:50:29Z  
Registry Expiry Date: 2027-06-14T07:50:29Z  
Registrar: Gandi SAS  
Registrar IANA ID: 81  
Registrar Abuse Contact Email: abuse@support.gandi.net  
Registrar Abuse Contact Phone: +33.170377661  
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited  
Name Server: NS-105-A.GANDI.NET  
Name Server: NS-11-B.GANDI.NET  
Name Server: NS-140-C.GANDI.NET  
DNSSEC: unsigned  
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/  
>>> Last update of whois database: 2026-03-04T11:54:46Z <<<  
  
For more information on Whois status codes, please visit https://icann.org/epp  
  
NOTICE: The expiration date displayed in this record is the date the  
registrar's sponsorship of the domain name registration in the registry is  
currently set to expire. This date does not necessarily reflect the expiration  
date of the domain name registrant's agreement with the sponsoring  
registrar. Users may consult the sponsoring registrar's Whois database to  
view the registrar's reported date of expiration for this registration.
```

This information confirms domain ownership and hosting infrastructure details.

```
kali@kali: /  
The Registry database contains ONLY .COM, .NET, .EDU domains and  
Registrars.  
Domain Name: vulnweb.com  
Registry Domain ID: 1602006391_DOMAIN_COM-VRSN  
Registrar WHOIS Server: whois.gandi.net  
Registrar URL: http://www.gandi.net  
Updated Date: 2025-11-17T09:34:23Z  
Creation Date: 2010-06-14T05:50:29Z  
Registrar Registration Expiration Date: 2027-06-14T07:50:29Z  
Registrar: GANDI SAS  
Registrar IANA ID: 81  
Registrar Abuse Contact Email: abuse@support.gandi.net  
Registrar Abuse Contact Phone: +33.170377661  
Reseller:  
Domain Status: clientTransferProhibited http://www.icann.org/epp#clientTransferProhibited  
Domain Status:  
Domain Status:  
Domain Status:  
Domain Status:  
Domain Status:  
Registry Registrant ID: REDACTED FOR PRIVACY  
Registrant Name: REDACTED FOR PRIVACY  
Registrant Organization: Invicti Security Limited  
Registrant Street: REDACTED FOR PRIVACY  
Registrant City: REDACTED FOR PRIVACY  
Registrant State/Province:  
Registrant Postal Code: REDACTED FOR PRIVACY  
Registrant Country: MT  
Registrant Phone: REDACTED FOR PRIVACY  
Registrant Phone Ext:  
Registrant Fax: REDACTED FOR PRIVACY  
Registrant Fax Ext:
```


HTTP Header Analysis:

HTTP headers were analysed to identify server configuration details and technology information disclosure.

Command Used:

curl -I <http://testphp.vulnweb.com>

Observed Headers:

Server: nginx/1.19.0

X-Powered-By: PHP/5.6.40

```
(kali㉿kali)-[//]  
$ curl -I http://testphp.vulnweb.com  
HTTP/1.1 200 OK  
Server: nginx/1.19.0  
Date: Wed, 04 Mar 2026 12:03:57 GMT  
Content-Type: text/html; charset=UTF-8  
Connection: keep-alive  
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
```

Exposing server software and technology versions can help attackers identify known vulnerabilities associated with those technologies.

Network Scanning:

Network scanning was conducted using Nmap to identify open ports and active services running on the server.

Command Used: *nmap testphp.vulnweb.com*

Result:

Open Port Detected: 80/tcp – HTTP

```
(kali㉿kali)-[/]
$ nmap testphp.vulnweb.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-04 09:30 -0500
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.020s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 52.29 seconds
```

This indicates that the web server is publicly accessible via HTTP.

Service Version Detection:

A service detection scan was performed to identify the version of the running web service.

Command Used: *nmap -sV testphp.vulnweb.com*

Result:

Service detected: HTTP service running on nginx version 1.19.0.

```
(kali㉿kali)-[/]
$ nmap -sV testphp.vulnweb.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-04 09:34 -0500
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.033s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.19.0

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 91.90 seconds
```

Identifying service versions allows security analysts to determine whether known vulnerabilities exist for the detected software.

Vulnerability Script Scan:

An automated vulnerability detection script was executed using Nmap to identify potential weaknesses in the target web application.

Command Used: *nmap --script vuln testphp.vulnweb.com*

```
(kali㉿kali)-[/]
└─$ nmap --script vuln testphp.vulnweb.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-04 09:45 -0500
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.054s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-cross-domain-policy:
|   VULNERABLE:
|   Cross-domain and Client Access policies.
|   State: VULNERABLE
|   A cross-domain policy file specifies the permissions that a web client such as Java, Adobe Flash, Adobe Reader,
|   etc. use to access data across different domains. A client access policy file is similar to cross-domain policy
|   but is used for M$ Silverlight applications. Overly permissive configurations enables Cross-site Request
|   Forgery attacks, and may allow third parties to access sensitive data meant for the user.
|   Check results:
|   /crossdomain.xml:
|
|   </cross-domain-policy>
|   /clientaccesspolicy.xml:
```

The scan detected potential security issues including:

- Cross-domain policy configuration issues
- Possible administrative directories exposed
- Potential Cross-Site Request Forgery (CSRF) weaknesses

```
</access-policy>
Extra information:
  Trusted domains:*

References:
  https://www.adobe.com/devnet/articles/crossdomain_policy_file_spec.html
  http://gursevkakra.blogspot.com/2013/08/bypassing-same-origin-policy-with-flash.html
  http://acunetix.com/vulnerabilities/web/insecure-clientaccesspolicy-xml-file
  http://sethsec.blogspot.com/2014/03/exploiting-misconfigured-crossdomainxml.html
  https://www.owasp.org/index.php/Test_RIA_cross_domain_policy_%280TG-CONFIG-008%29
  https://www.adobe.com/devnet-docs/acrobatetk/tools/AppSec/CrossDomain_PolicyFile_Specification.pdf
|_http-sql-injection: ERROR: Script execution failed (use -d to debug)
http-enum:
  /admin/: Possible admin folder
  /login.php: Possible admin folder
  /clientaccesspolicy.xml: Microsoft Silverlight crossdomain policy
  /crossdomain.xml: Adobe Flash crossdomain policy
http-csrf:
Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=testphp.vulnweb.com
Found the following possible CSRF vulnerabilities:

  Path: http://testphp.vulnweb.com:80/
  Form id:
  Form action: search.php?test=query

  Path: http://testphp.vulnweb.com:80/artists.php
  Form id:
  Form action: search.php?test=query
```

Remediation Recommendations:

1. Cross-Domain Policy Configuration

Restrict cross-domain access by properly configuring cross-domain policy files such as **crossdomain.xml** and **clientaccesspolicy.xml**. Only trusted domains should be granted access permissions.

Recommended actions:

- Avoid using wildcard permissions (*) in cross-domain policies
- Limit access to trusted domains only
- Regularly review cross-domain policy configurations

2. Exposed Administrative Directories

Administrative paths such as /admin/ or /login.php may be discoverable through automated scans.

Recommended actions:

- Restrict access to administrative directories using authentication mechanisms
- Implement IP-based access restrictions where possible
- Avoid exposing administrative endpoints publicly

3. Potential CSRF Vulnerabilities

Forms detected during the scan may lack adequate protection against **Cross-Site Request Forgery (CSRF)** attacks.

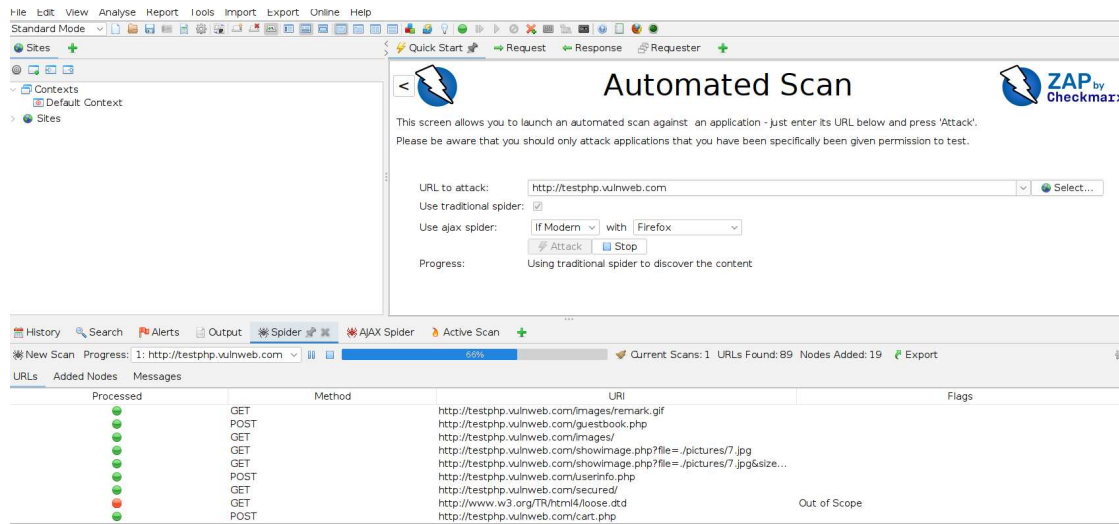
Recommended actions:

- Implement unique CSRF tokens for sensitive requests
- Validate tokens on the server side before processing requests
- Ensure forms and sensitive actions require authenticated sessions

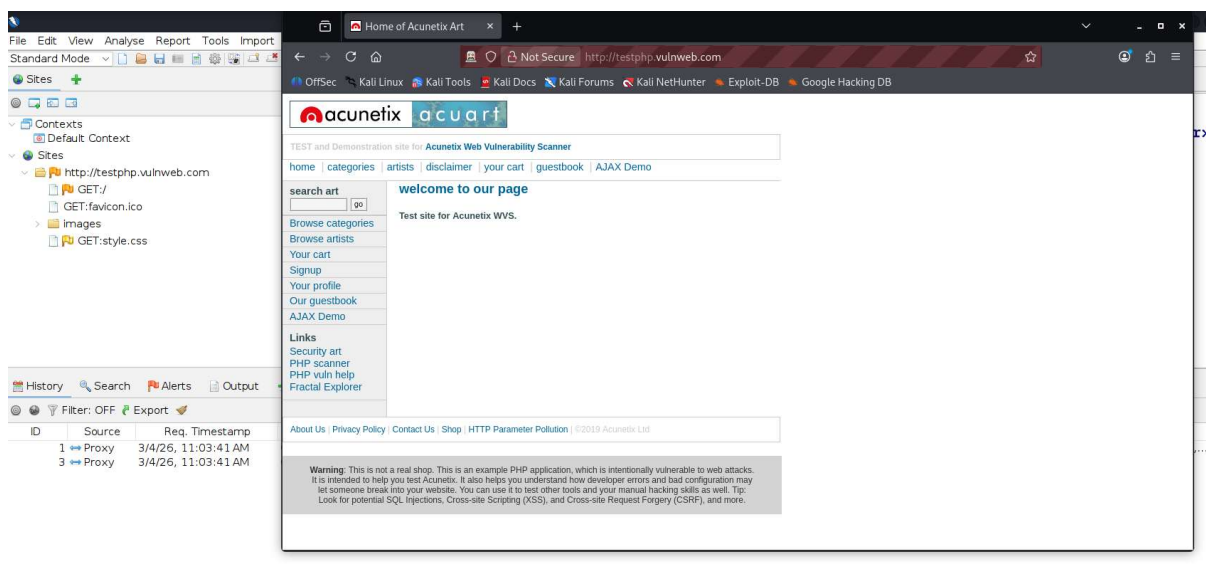
Web Application Vulnerability Scan:

OWASP ZAP was used to perform an automated vulnerability scan on the target web application.

OWASP Zed Attack Proxy (ZAP) was used to scan the application, identify endpoints, and analyse HTTP responses to detect potential vulnerabilities.



OWASP ZAP is a security scanner and proxy tool that helps identify vulnerabilities such as XSS, SQL Injection, CSRF, and misconfigurations in web applications.



Vulnerability Findings Summary:

The web application scan performed using **OWASP ZAP** identified multiple security issues within the target web application. The findings were categorized according to their **severity level** based on the potential impact they may have on the application's security. Severity levels indicate how critical a vulnerability may be if exploited by an attacker.

The scan results revealed a combination of **high, medium, and low severity issues**, mainly related to security misconfigurations, missing security headers, and improper cookie handling.

Risk Summary

The following table summarizes the vulnerabilities detected during the assessment:

Risk Level	No of Findings	Description
High	1	Issues that may lead to serious security compromise or sensitive data exposure.
Medium	3	Security weaknesses that could increase the attack surface or enable certain attacks.
Low	2	Minor security misconfigurations or information disclosures with limited impact.

The screenshot displays the OWASP ZAP Alerts panel, which lists 36 identified security issues. The issues are categorized by severity: High (1), Medium (3), and Low (2). The list includes various misconfigurations and missing security headers, such as 'Hash Disclosure - BCrypt', 'Absence of Anti-CSRF Tokens', 'Application Error Disclosure', 'CSP: Failure to Define Directive with No Failb', 'CSP: Wildcard Directive', 'CSP: script-src unsafe-inline', 'CSP: style-src unsafe-inline', 'Content Security Policy (CSP) Header Not Set', 'Cross-Domain Misconfiguration', and 'Missing Anti-clickjacking Header'. The panel also provides instructions on how to manually add or edit alerts.

History Search Alerts Output +

Alerts (36)

- Hash Disclosure - BCrypt (4)
- Absence of Anti-CSRF Tokens (Systemic)
- Application Error Disclosure
- CSP: Failure to Define Directive with No Failb
- CSP: Wildcard Directive
- CSP: script-src unsafe-inline
- CSP: style-src unsafe-inline
- Content Security Policy (CSP) Header Not Set
- Cross-Domain Misconfiguration (18)
- Missing Anti-clickjacking Header (Systemic)

Full details of any selected alert will be displayed here.

You can manually add alerts by right clicking on the relevant line in the history and selecting 'Add alert'.

You can also edit existing alerts by double clicking on them.

History Search Alerts Output +

- X-Content-Type-Options Header Missing (Syst
- Authentication Request Identified
- Charset Mismatch (Header Versus Meta Cont
- Information Disclosure - Information in Browsi
- Information Disclosure - Information in Browsi
- Information Disclosure - Sensitive Information
- Information Disclosure - Suspicious Comment
- Loosely Scoped Cookie (7)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (13)
- Retrieved from Cache (Systemic)
- Session Management Response Identified (9)

Full details of any selected alert will be displayed here.

You can manually add alerts by right clicking on the relevant line in the history and selecting 'Add alert'.

You can also edit existing alerts by double clicking on them.

Finding 1 — Hash Disclosure (BCrypt)

Risk Level - **High**

Description

The automated vulnerability scan detected that the application exposes a **BCrypt hash value** in the server response. Hash disclosure occurs when hashed credentials or sensitive values are visible within application responses, logs, or other accessible outputs.

Although hashed values are encrypted representations of data, attackers may attempt to perform **offline brute-force or dictionary attacks** to recover the original values, especially if weak passwords are used.



In this case, the scan detected a hash value similar to: \$2a\$10\$... which indicates a **BCrypt hashed value** exposed by the web application.

Business Impact

- Attackers may attempt **offline password cracking** using specialized tools
- Weak passwords may be recovered from exposed hashes
- Compromised credentials may allow **unauthorized access** to user accounts
- Sensitive authentication data may be exposed to unauthorized parties

Recommendation

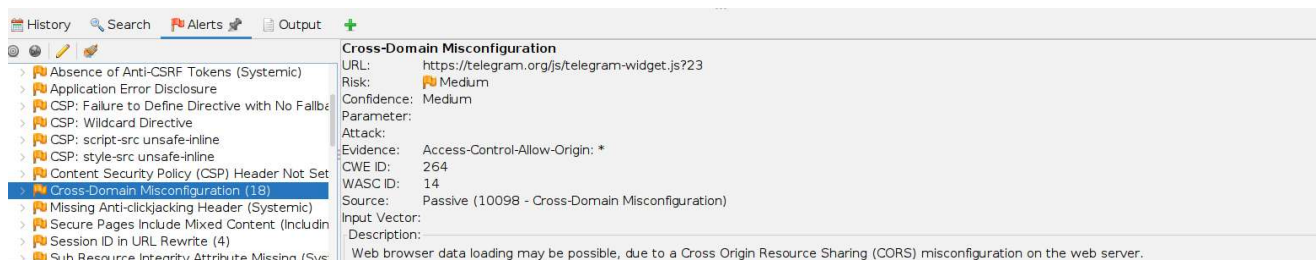
- Avoid exposing hashed credentials or sensitive authentication data in application responses
- Ensure password hashes are stored securely on the **server side only**
- Implement proper output filtering to prevent accidental disclosure of sensitive values

Finding 2 — Cross-Domain Misconfiguration (CORS)

Risk Level: **Medium**

Description

The automated security scan identified a Cross-Origin Resource Sharing (CORS) misconfiguration within the application. The server response includes a permissive header configuration allowing requests from any origin.



The detected response header contains: Access-Control-Allow-Origin: *

This configuration allows resources on the server to be accessed by **any external domain**, which may unintentionally expose sensitive information or application functionality.

Business Impact

- Malicious websites may interact with the application from unauthorized domains
- Sensitive data may be accessed by untrusted external sources
- Attackers may exploit cross-origin requests to perform unauthorized actions
- Increased risk of **data leakage or cross-site attacks**

Recommendation

- Avoid using wildcard (*) values in the **Access-Control-Allow-Origin** header
- Restrict cross-origin access to **trusted domains only**
- Implement strict CORS policies defining allowed origins, methods, and headers
- Regularly review server configurations to ensure secure resource sharing

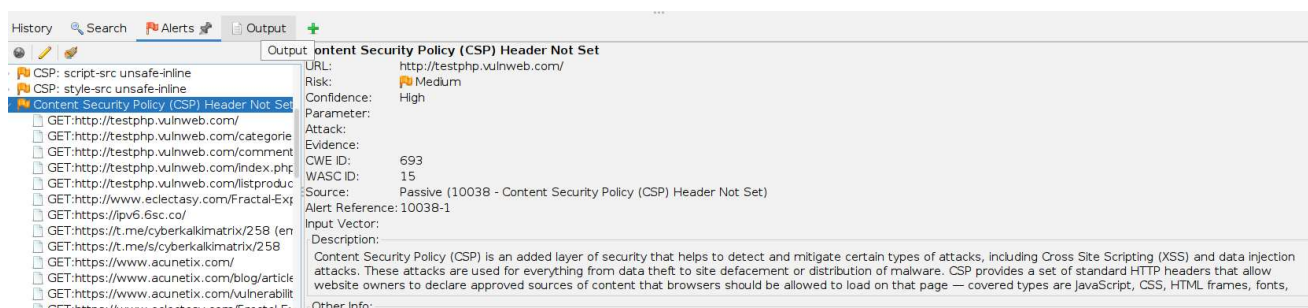
Finding 3 — Missing Anti-Clickjacking Header

Risk Level: **Medium**

Description

The web application does not include an **X-Frame-Options** header or an equivalent **Content Security Policy (CSP)** directive.

Without these protections, the site can be embedded in malicious webpages, making it vulnerable to **clickjacking attacks** where users may unknowingly perform unintended actions.



The screenshot shows a security tool interface with a 'History' tab selected. The 'Output' pane displays a finding titled 'Content Security Policy (CSP) Header Not Set'. The finding details include:

- URL: http://testphp.vulnweb.com/
- Risk: Medium
- Confidence: High
- Parameter:
- Attack:
- Evidence: CWE ID: 693, WASC ID: 15
- Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
- Alert Reference: 10038-1
- Input Vector:
- Description: Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, and images.
- Other Info:

Business Impact

If clickjacking protections are missing, attackers may:

- Trick users into clicking hidden elements on the application
- Perform unauthorized actions on behalf of users
- Manipulate user interactions through malicious overlays
- Compromise sensitive user actions such as login or account changes

Recommendation

Implement one of the following security headers.

Add the X-Frame-Options header: X-Frame-Options: DENY or

X-Frame-Options: SAMEORIGIN

Alternatively, use a Content Security Policy directive:

Content-Security-Policy: frame-ancestors 'none'

This prevents the application from being embedded within unauthorized external frames

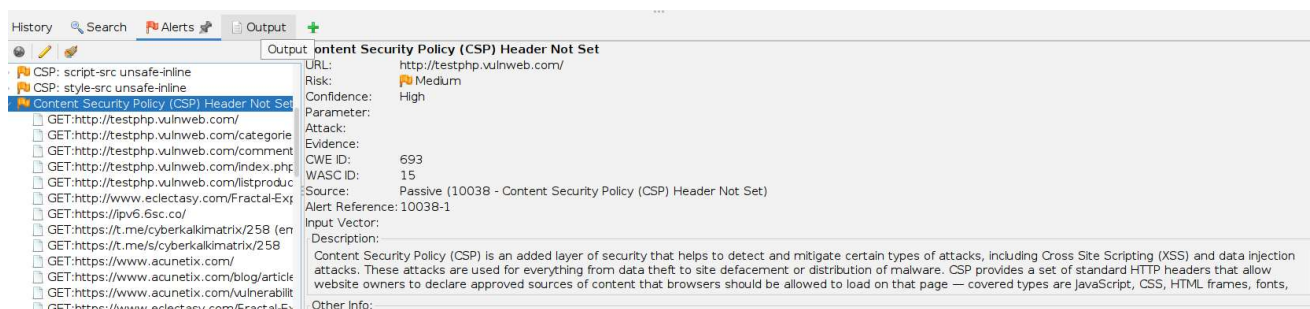
Finding 4 — Content Security Policy Header Not Set

Risk Level: **Medium**

Description

The scan detected that the web application does not include a **Content Security Policy (CSP)** header in its HTTP responses. CSP is a security mechanism that helps prevent attacks such as **Cross-Site Scripting (XSS)** and data injection by restricting which sources of content are allowed to load in the browser.

Without a CSP policy, the browser cannot enforce restrictions on scripts, styles, or other resources loaded by the web application.



Business Impact

- Increased risk of **Cross-Site Scripting (XSS)** attacks
- Malicious scripts may execute in the user's browser
- Sensitive user data may be exposed or stolen

Recommendation

Implement a **Content Security Policy header** to control allowed resource sources.

Example configuration:

Content-Security-Policy: default-src 'self';

Additional measures:

- Restrict scripts, styles, and external resources
- Allow content only from trusted domains

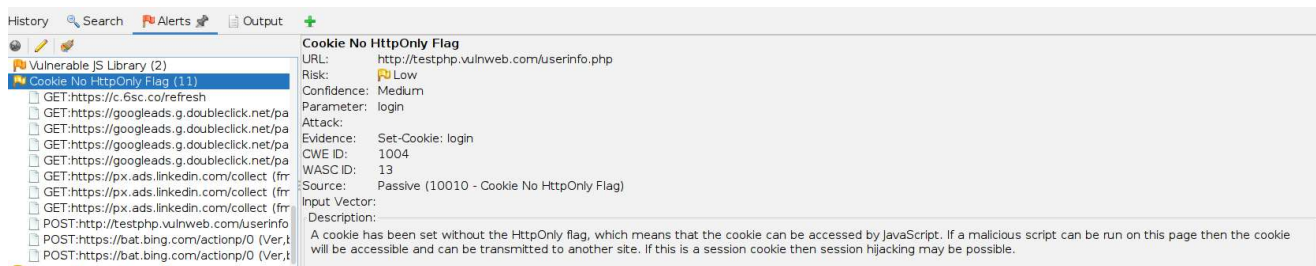
Finding 5 — Cookie Without HttpOnly Flag

Risk Level: **Low**

Description

The scan identified that a cookie used by the web application is set **without the HttpOnly attribute**. The HttpOnly flag prevents client-side scripts such as JavaScript from accessing cookie values.

Without this protection, cookies may be accessible through browser scripts if a Cross-Site Scripting (XSS) vulnerability exists.



Business Impact

Cookies without the HttpOnly flag may introduce several risks:

- Client-side scripts can access sensitive cookies
- Attackers could steal session cookies using **Cross-Site Scripting (XSS)** attacks
- Stolen cookies may allow attackers to **hijack user sessions**
- Unauthorized users may gain access to authenticated sessions

Recommendation

Enable the **HttpOnly flag** for all sensitive cookies to prevent access from client-side scripts.

Example:

Set-Cookie: session_id=xyz123; HttpOnly; Secure

Additionally:

- Use the **Secure** attribute for HTTPS transmission
- Implement **SameSite cookie policies**

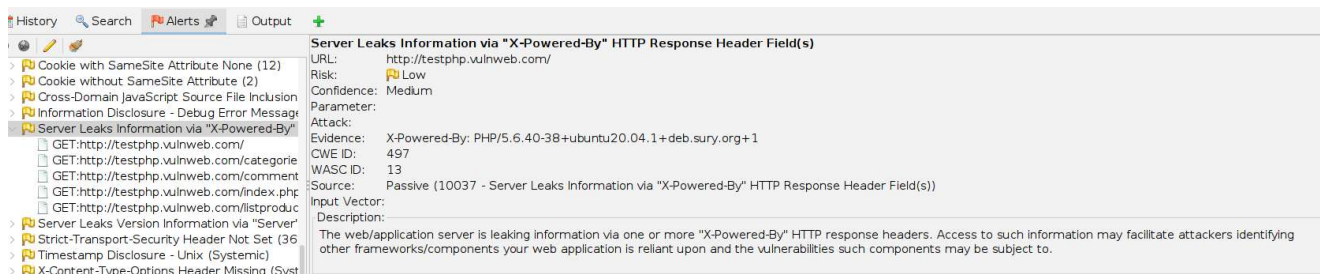
Finding 6 — Server Information Disclosure

Risk Level: **Low**

Description

The scan identified that the web server exposes technology details through the **X-Powered-By HTTP response header**. In this case, the header reveals the server is running **PHP/5.6.40**.

Exposing server technologies may provide attackers with useful information about the application stack and its underlying components.



The screenshot shows a security scanner interface. On the left, a list of findings includes 'Cookie with SameSite Attribute None (12)', 'Cookie without SameSite Attribute (2)', 'Cross-Domain JavaScript Source File Inclusion', 'Information Disclosure - Debug Error Message', 'Server Leaks Information via "X-Powered-By"', 'Server Leaks Version Information via "Server"', 'Strict-Transport-Security Header Not Set (36)', 'Timestamp Disclosure - Unix (Systemic)', and 'X-Content-Type-Options Header Missing (Syst)'. The 'Server Leaks Information via "X-Powered-By"' finding is selected. The right pane shows details for this finding: URL: http://testphp.vulnweb.com/, Risk: Low, Confidence: Medium, Parameter: (empty), Attack: (empty), Evidence: X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1, CWE ID: 497, WASC ID: 13, Source: Passive (10037 - Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)), Input Vector: (empty), and Description: The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.

Business Impact

- Attackers can identify technologies used by the server
- Known vulnerabilities in the exposed software version may be targeted
- Increases the likelihood of **targeted attacks**

Recommendation

Remove or hide server technology information from HTTP response headers.

Example secure configuration:

X-Powered-By: (disabled)

Additionally:

- Disable unnecessary response headers
- Keep server software **updated and patched**
- Implement secure server configurations

Summary of Findings

Sl. No	Findings	Risk Level
1	Hash Disclosure (BCrypt)	High
2	Cross-Domain Misconfiguration (CORS)	Medium
3	Missing Anti-Clickjacking Header	Medium
4	Content Security Policy Header Not Set	Medium
5	Cookie Without HttpOnly Flag	Low
6	Server Information Disclosure	Low

Overall Security Recommendations

The following improvements are recommended to enhance the security of the web application.

1. Implement Security Headers

Configure security headers such as Content Security Policy (CSP) and X-Frame-Options to protect against attacks like XSS and clickjacking.

2. Restrict CORS Policies

Avoid using permissive configurations such as Access-Control-Allow-Origin:

*. Allow access only from trusted domains.

3. Secure Cookie Settings

Enable HttpOnly, Secure, and SameSite attributes for session cookies to improve session security.

4. Prevent Information Disclosure

Remove unnecessary server headers such as X-Powered-By to prevent exposure of server technologies.

5. Perform Regular Security Testing : Conduct periodic vulnerability assessments and security reviews to identify and fix new security issues.

Assessment Limitations

This security assessment was conducted using non-intrusive and passive testing techniques. The scope of the assessment was limited to publicly accessible components of the target web application.

The following activities were not performed during the assessment:

- Exploitation of identified vulnerabilities
- Authentication bypass attempts
- Denial-of-Service (DoS) testing
- Data extraction or modification

As a result, additional vulnerabilities may exist that were not detectable through passive analysis alone.

Conclusion

This vulnerability assessment identified several security weaknesses within the target web application, including misconfigured security headers, cookie security issues, and information disclosure through server responses.

Although most findings were classified as low to medium risk, they highlight areas where security controls can be improved. Implementing the recommended mitigation measures will help strengthen the overall security posture of the application.

Regular security testing and adherence to secure development practices are essential to ensure continued protection against evolving cyber threats.